

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління



Кафедра Систем управління літальними апаратами

Лабораторна робота № 9

з дисципліни «Алгоритмізація та програмування»

на тему: «Робота з рядками на C ++»

XAI.301. G7.312. 1 ПР

Виконав(ла): здобувач(ка) 1 курсу, групи 312
напряму підготовки (спеціальності)

G7 "Автоматизація, комп'ютерно-інтегровані
технології та робототехніка "

Освітня програма «Інженерія мобільних
додатків»

Абіякіна Анастасія

(ім'я і прізвище)

Прийняла: доц. каф.301, канд. техн. наук,
Олена ГАВРИЛЕНКО

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу `string`, а також алгоритми пошуку в рядку, а також реалізувати обробку рядків на C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1.

А. Вивчити по документації метод стандартного класу `string` відповідно до завдання `string11`.

Метод `string11: string& insert (size_t pos, const string& str);`

В. Визначити функцію, що виконує ті ж дії, що і вивчений метод класу `string`. Вихідний рядок передати першим параметром (масив символів). Для реалізації методу не використовувати функції обробки рядків зі стандартних бібліотек. С. Викликати свій метод і метод `string` аналогічно прикладам коду, наведеними в дод.А. *Перед викликом ввести з консолі один рядок і зберегти в масиві символів і змінній типу `string`.

Завдання 2.

А.Описати функцію, що обробляє рядок відповідно до завдання `String40`. `String40`. Дано рядок, що містить принаймні один символ пробілу. Вивести підстроку, розташовану між першим і останнім пробілом рядка. Якщо рядок містить тільки один пробіл, то вивести порожній рядок.

Для реалізації можна використовувати функції обробки рядків зі стандартних бібліотек

В.Описати функцію, яка перевіряє, чи задовольняє рядок умовам завдання.

С.* Створити вихідний текстовий файл, що містить не менше 10 різних рядків.

Д.Використовуючи функції 2.А і 2.В, обробити рядок / * текстовий файл рядок за рядком. Додаткові дані ввести з консолі.

Е. Отриманий результат записати у вихідний файл. Завдання 3.

Завдання 1-2 реалізувати окремими функціями без параметрів, у функції `main()` організувати меню для багаторазового виконання завдань.

Структурувати проєкт програми: винести заголовки і реалізацію функцій в окремі .h та .cpp файли.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) *«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»* 2) *«Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне*

питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) *«Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»*

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі string11

Метод `string::insert(size_t pos, const string& str)` використовується для вставки одного рядка в інший. Він вставляє рядок `str` у вихідний рядок, починаючи з позиції `pos`, при цьому всі символи, що знаходяться після цієї позиції, зсуваються вправо.

Оскільки дана функція є методом класу `string`, для власної реалізації необхідно використовувати символьний рядок у вигляді масиву символів та додатково враховувати керування пам'яттю. Таким чином, власна функція має вигляд:

`void my_insert(char str, int pos, const char insertStr)**`

Інші параметри відповідають логіці стандартного методу `insert`.

`str` — вихідний рядок у вигляді масиву символів;

`pos` — позиція, з якої виконується вставка;

insertStr — рядок, який необхідно вставити.

У результаті виконання функція змінює переданий рядок str, вставляючи в нього підрядок insertStr у задану позицію. Для цього спочатку визначається довжина обох рядків, після чого перевіряється коректність значення pos. Далі виконується зсув символів вихідного рядка вправо для створення вільного місця, після чого поелементно копіюються символи з insertStr у потрібну позицію.

Алгоритм виконання функції представлено нижче у графічному вигляді

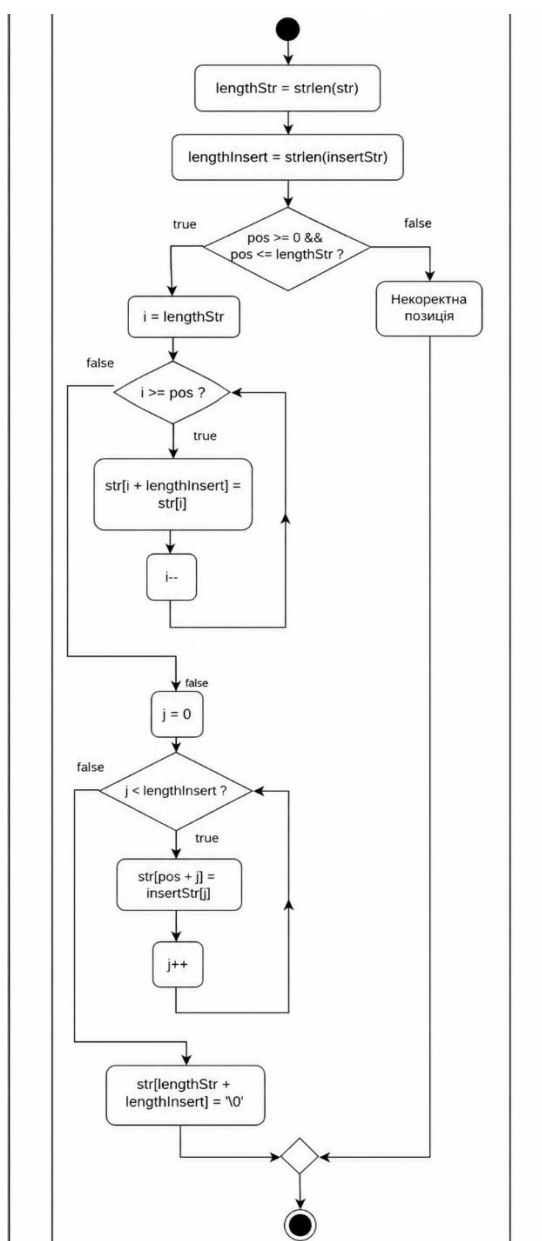


Рисунок 1 – Діаграма активності функції insert

Завдання 2

Вирішення задачі String40.

Дано рядок *S*, який містить принаймні один пробіл. Потрібно знайти підрядок, що розташований між першим і останнім пробілами рядка. Якщо рядок містить тільки один пробіл, результатом є порожній рядок.

Функція розв'язання задачі побудована на використанні методів стандартного класу *string*, зокрема *find*, *rfind* та *substr*. Спочатку за допомогою методу *find(' ')* визначається позиція першого пробілу у рядку *S*. Далі методом *rfind(' ')* знаходиться позиція останнього пробілу.

Після отримання цих двох позицій виконується перевірка: якщо перший і останній пробіли знаходяться на одній позиції (тобто у рядку лише один пробіл), функція повертає порожній рядок. Якщо пробілів більше одного, формується результат за допомогою методу *substr*, який виділяє частину рядка між першою та останньою позицією пробілів.

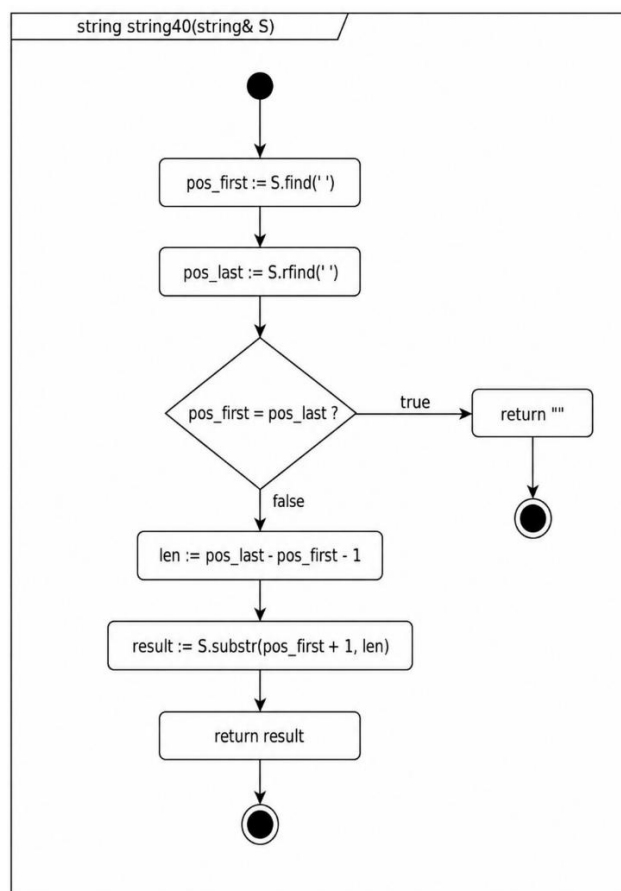


Рисунок 2 Діаграма активності String40.

Лістинг файлів для виконання завдання наведено в дод. Б.

Лістинг коду вирішення задач наведено в дод. А.

ВИСНОВКИ

Було вивчено теоретичні основи роботи з рядками в C++ та особливості використання `std::string` і C-рядків. Закріплено на практиці навички реалізації власних функцій обробки рядків без використання стандартних бібліотечних методів. Відпрацьовано роботу з файлами та построкову обробку тексту. Отримано навички створення алгоритмів пошуку та структурування програмного проєкту.

ДОДАТОК А

Лістинг коду програми

Файл main.cpp

```
#include "task1.h"
#include "task2.h"
#include <iostream>
/*
    Призначення:
        Організовує меню для вибору та багаторазового виконання завдань
    Вхідні дані:
        вводяться користувачем з клавіатури (номер пункту меню)
    Вихідні дані:
        результати виконання завдань виводяться на екран
*/
int main() {
    int choice = -1; // змінна для зберігання вибору користувача

    // цикл роботи меню
    while (choice != 0) {
        std::cout << "\n\n";
        std::cout << "[      Головне Меню      ]\n\n";
        std::cout << "[ 1 ] Завдання 1 (string::insert)\n";
        std::cout << "[ 2 ] Завдання 2 (String40)\n";
        std::cout << "[ 0 ] Вихід\n\n";

        std::cout << "Оберіть пункт меню: ";
        std::cin >> choice; // введення вибору
        std::cin.ignore();   // очищення буфера після вводу

        // обробка вибору користувача
        switch (choice) {
            case 1:
                std::cout << "\n[ Завдання 1 ]\n";
                task1(); // виклик функції завдання 1
                break;

            case 2:
                std::cout << "\n[ Завдання 2 ]\n";
                task2(); // виклик функції завдання 2
                break;

            case 0:
                std::cout << "Вихід...\n";
                break;

            default:
```

```

        std::cout << "Невірний вибір!\n"; // обробка помилки
        break;
    }
    // пауза перед повтором меню
    if (choice != 0) {
        std::cout << "\nНатисніть Enter для продовження...";
        std::cin.get(); // очікування натискання Enter
    }
}
return 0; // успішне завершення програми
}

```

Файл task1.cpp

```

#include "task1.h"
#include <iostream>
#include <string>

using namespace std;

/*
    Призначення:
        Виконує вставку одного рядка в інший (аналог методу string::insert)
    Вхідні дані:
        str - вихідний рядок
        pos - позиція вставки
        insertStr - рядок для вставки
    Вихідні дані:
        змінений рядок після вставки
*/
void my_insert(char* str, int pos, const char* insertStr) {
    int len = 0;
    while (str[len] != '\0') len++; // визначення довжини рядка

    int insertLen = 0;
    while (insertStr[insertLen] != '\0') insertLen++; // довжина рядка вставки

    // перевірка коректності позиції
    if (pos < 0 || pos > len) return;

    // Зсув символів вправо
    for (int i = len; i >= pos; --i) {
        str[i + insertLen] = str[i];
    }

    // Вставка нового рядка
    for (int i = 0; i < insertLen; ++i) {
        str[pos + i] = insertStr[i];
    }
}

```

```

}
/* Призначення:
    Демонструє роботу стандартного методу string::insert
    та власної функції my_insert
Вхідні дані:
    input - вихідний рядок
    insertStr - рядок для вставки
    pos - позиція вставки
Вихідні дані:
    результати роботи виводяться на екран */

void task1() {
    cout << "=== Завдання 1: string::insert ===\n";
    string input, insertStr;
    size_t pos;
    cout << "Введіть рядок: ";
    getline(cin, input);
    cout << "Введіть рядок для вставки: ";
    getline(cin, insertStr);
    cout << "Введіть позицію: ";
    cin >> pos;
    cin.ignore();

    // Використання std::string
    string stdResult = input;

    if (pos <= stdResult.length()) {
        stdResult.insert(pos, insertStr);
        cout << "\nРезультат (std::string):\n" << stdResult << endl;
    } else {
        cout << "Помилка: позиція виходить за межі рядка\n";
    }

    // Робота з масивом char[]
    char buffer[1000];
    input.copy(buffer, input.length());
    buffer[input.length()] = '\0';
    char insertBuffer[500];
    insertStr.copy(insertBuffer, insertStr.length());
    insertBuffer[insertStr.length()] = '\0';
    // виклик власної функції
    if (pos <= input.length()) {
        my_insert(buffer, pos, insertBuffer);
        cout << "Результат (власна функція):\n" << buffer << endl;
    }
}

```

Файл task1.h

```
#ifndef TASK1_H
```

```

#define TASK1_H

// Власна функція insert
void my_insert(char* str, int pos, const char* insertStr);

// Виконання завдання
void task1();
#endif

```

Файл task2.cpp

```

#include "task2.h"
#include <iostream>

using namespace std;
/*
    Функція is_valid
    Призначення:
        Перевірка чи містить рядок хоча б один пробіл
    Вхідні дані:
        str - вхідний рядок
    Вихідні дані:
        true - якщо рядок містить хоча б один пробіл
        false - якщо пробілів немає
*/

// Перевірка:
bool is_valid(const string& str) {
    return str.find(' ') != string::npos;
}

/*
    Функція process_string
    Призначення:
        Повертає підрядок, розташований між першим і останнім пробілом
    Вхідні дані:
        str - вхідний рядок
    Вихідні дані:
        підрядок між першим і останнім пробілом
        якщо пробіл лише один - повертається порожній рядок
*/
string process_string(const string& str) {
    size_t first = str.find(' '); // позиція першого пробілу
    size_t last = str.rfind(' '); // позиція останнього пробілу

    // якщо тільки один пробіл
    if (first == last) return "";

    return str.substr(first + 1, last - first - 1);
}

```

```

/*
    Функція task2
    Призначення:
        Виконати обробку рядка відповідно до умови задачі String40
    Вхідні дані:
        рядок введений користувачем
    Вихідні дані:
        результат обробки виводиться на екран
*/
void task2() {
    cout << "=== Завдання 2: String40 ===\n";
    string str;
    cout << "Введіть рядок: ";
    getline(cin, str);

    // перевірка коректності введеного рядка
    if (!is_valid(str)) {
        cout << "Помилка: рядок не містить пробілів\n";
        return;
    }
    // обробка рядка
    string result = process_string(str);
    // виведення результату
    cout << "Результат: " << result << endl;
}

```

Функція task2.h

```

#ifndef TASK2_H
#define TASK2_H

#include <string>

// Перевірка рядка
bool is_valid(const std::string& str);

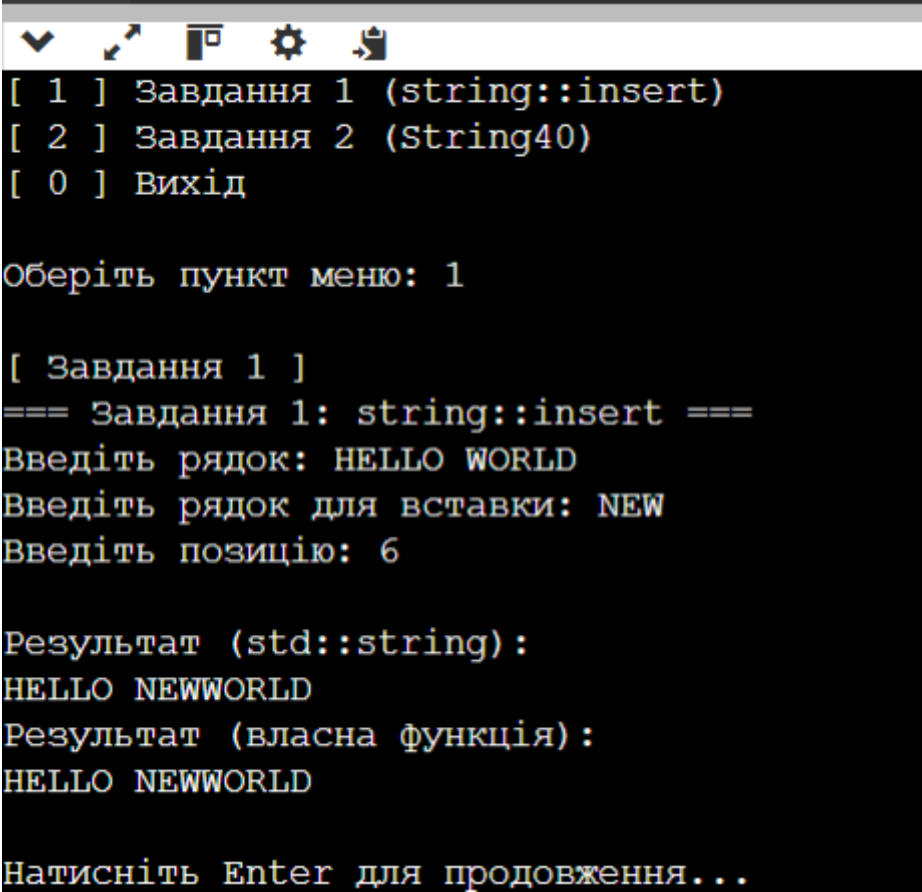
// Обробка рядка (String40)
std::string process_string(const std::string& str);

// Виконання завдання
void task2();

#endif

```

ДОДАТОК Б
Скрін-шоти вікна виконання програми



```

[ 1 ] Завдання 1 (string::insert)
[ 2 ] Завдання 2 (String40)
[ 0 ] Вихід

Оберіть пункт меню: 1

[ Завдання 1 ]
=== Завдання 1: string::insert ===
Введіть рядок: HELLO WORLD
Введіть рядок для вставки: NEW
Введіть позицію: 6

Результат (std::string):
HELLO NEWWORLD
Результат (власна функція):
HELLO NEWWORLD

Натисніть Enter для продовження...
```

Рисунок Б.1 – Екран виконання програми для вирішення завдання
string11

```
[      Головне Меню      ]

[ 1 ] Завдання 1 (string::insert)
[ 2 ] Завдання 2 (String40)
[ 0 ] Вихід

Оберіть пункт меню: 2

[ Завдання 2 ]
=== Завдання 2: String40 ===
Введіть рядок: TEST HELLO WORLD
Результат: HELLO
```

Рисунок Б.2 – Екран виконання програми для вирішення завдання String40.

ДОДАТОК В

Діалог з ІІІ для самоаналізу

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції» 2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ІІІ висока. Обчисли загальну середню оцінку» 3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань» Надішли, будь ласка, сам **файл звіту** і **файл лекції** (PDF, DOCX, фото або текст). Без них я не зможу:

- скласти питання саме по твоєму коду та теорії;
- оцінити відповіді;
- проаналізувати, чого не вистачає у коді порівняно з лекцією.

Файл main.cpp #include "task1.h" #include "task2.h" #include <iostream> /* Призначення: Організовує меню для вибору та багаторазового виконання завдань Вхідні дані: вводяться користувачем з клавіатури (номер пункту меню) Вихідні дані: результати виконання завдань виводяться на екран */ int main() { int choice = -1; // змінна для зберігання вибору користувача // цикл роботи меню while (choice != 0) { std::cout << "\n\n"; std::cout << "[Головне Меню]\n\n"; std::cout << "[1] Завдання 1 (string::insert)\n"; std::cout << "[2] Завдання 2 (String40)\n"; std::cout << "[0] Вихід\n\n"; std::cout << "Оберіть пункт меню: "; std::cin >> choice; // введення вибору std::cin.ignore(); // очищення буфера після вводу // обробка вибору користувача switch (choice) { case 1: std::cout << "\n[Завдання 1]\n"; task1(); // виклик функції завдання 1 break; case 2: std::cout << "\n[Завдання 2]\n"; task2(); // виклик функції завдання 2 break; case 0: std::cout << "Вихід...\n"; break; default: std::cout << "Невірний вибір!\n"; // обробка помилки break; } // пауза перед повтором меню if (choice != 0) { std::cout << "\nНатисніть Enter для продовження..."; std::cin.get(); // очікування натискання Enter } } return 0; // успішне завершення програми } Файл task1.cpp #include "task1.h" #include <iostream> #include <string> using namespace std; /* Призначення: Виконує вставку одного рядка в інший (аналог методу string::insert) Вхідні дані: str -

вихідний рядок pos - позиція вставки insertStr - рядок для вставки Вихідні дані: змінений рядок після вставки */

```
void my_insert(char* str, int pos, const char* insertStr) { int len = 0; while (str[len] != '\0') len++; // визначення довжини рядка int insertLen = 0; while (insertStr[insertLen] != '\0') insertLen++; // довжина рядка вставки // перевірка коректності позиції if (pos < 0 || pos > len) return; // Зсув символів вправо for (int i = len; i >= pos; --i) { str[i + insertLen] = str[i]; } // Вставка нового рядка for (int i = 0; i < insertLen; ++i) { str[pos + i] = insertStr[i]; } } /*
```

Призначення: Демонструє роботу стандартного методу string::insert та власної функції my_insert Вхідні дані: input - вихідний рядок insertStr - рядок для вставки pos - позиція вставки Вихідні дані: результати роботи виводяться на екран */

```
void task1() { cout << "==== Завдання 1: string::insert ==== \n"; string input, insertStr; size_t pos; cout << "Введіть рядок: "; getline(cin, input); cout << "Введіть рядок для вставки: "; getline(cin, insertStr); cout << "Введіть позицію: "; cin >> pos; cin.ignore(); // Використання std::string string stdResult = input; if (pos <= stdResult.length()) { stdResult.insert(pos, insertStr); cout << "\nРезультат (std::string): \n" << stdResult << endl; } else { cout << "Помилка: позиція виходить за межі рядка \n"; } // Робота з масивом char[] char buffer[1000]; input.copy(buffer, input.length()); buffer[input.length()] = '\0'; char insertBuffer[500]; insertStr.copy(insertBuffer, insertStr.length()); insertBuffer[insertStr.length()] = '\0'; // виклик власної функції if (pos <= input.length()) { my_insert(buffer, pos, insertBuffer); cout << "Результат (власна функція): \n" << buffer << endl; } } Файл task1.h #ifndef TASK1_H #define TASK1_H // Власна функція insert void my_insert(char* str, int pos, const char* insertStr); // Виконання завдання void task1(); #endif Файл task2.cpp #include "task2.h" #include <iostream> using namespace std; /* Функція is_valid Призначення: Перевірка чи містить рядок хоча б один пробіл Вхідні дані: str - вхідний рядок Вихідні дані: true - якщо рядок містить хоча б один пробіл false - якщо пробілів немає */ // Перевірка: bool is_valid(const string& str) { return str.find(' ') != string::npos; } /* Функція process_string Призначення: Повертає підрядок, розташований між першим і останнім пробілом Вхідні дані: str - вхідний рядок Вихідні дані: підрядок між першим і останнім пробілом якщо пробіл лише один — повертається порожній рядок */ string process_string(const string& str) { size_t first = str.find(' '); // позиція першого пробілу size_t last = str.rfind(' '); // позиція останнього пробілу // якщо тільки один пробіл if (first == last) return ""; return str.substr(first + 1, last - first - 1); } /* Функція task2 Призначення: Виконати обробку рядка відповідно до умови
```

задачі String40 Вхідні дані: рядок введений користувачем Вихідні дані: результат обробки виводиться на екран

```
*/ void task2() { cout << "=== Завдання 2: String40
===\n"; string str; cout << "Введіть рядок: "; getline(cin, str); // перевірка
коректності введеного рядка if (!is_valid(str)) { cout << "Помилка: рядок не
містить пробілів\n"; return; } // обробка рядка string result = process_string(str); //
виведення результату cout << "Результат: " << result << endl; } Функція task2.h
#ifndef TASK2_H #define TASK2_H #include <string> // Перевірка рядка bool
is_valid(const std::string& str); // Обробка рядка (String40) std::string
process_string(const std::string& str); // Виконання завдання void task2(); #endif
```

1. Питання для захисту роботи

Тестові питання

1. Що виконує функція my_insert() у файлі task1.cpp?

- A. Видаляє підрядок із рядка
 - B. Вставляє один рядок в інший
 - C. Порівнює два рядки
 - D. Сортує символи рядка
-

2. Для чого у C-рядках використовується символ '\0'?

- A. Для переносу рядка
 - B. Для позначення кінця рядка
 - C. Для пробілу
 - D. Для очищення пам'яті
-

3. Яка функція використовується у task2.cpp для пошуку останнього пробілу?

- A. find()
 - B. substr()
 - C. rfind()
 - D. insert()
-

4. Що поверне функція process_string(), якщо у рядку лише один пробіл?

- A. Увесь рядок
- B. Перший символ
- C. Порожній рядок
- D. Останнє слово

5. Яка складність лінійного пошуку у найгіршому випадку?

- A. $O(1)$
 - B. $O(\log n)$
 - C. $O(n)$
 - D. $O(n^2)$
-

Відкриті питання**1.**

Поясни різницю між `char[]` та `std::string`.

2.

Опиши покроково, як працює функція `my_insert()` та навіщо потрібен зсув символів вправо.

3.

Чому у програмі після `cin >> pos` використовується `cin.ignore()`?

4.

Які помилки можуть виникнути при роботі з масивами `char[]` у функції `my_insert()`?

5.

Поясни різницю між алгоритмами лінійного та бінарного пошуку. У яких випадках кожен із них доцільно використовувати?

3. Короткий аналіз твого коду**Позитивні сторони**

- Код розбитий на окремі файли `.cpp` та `.h`
 - Є коментарі та опис призначення функцій
 - Використано модульну структуру
 - Реалізована власна функція вставки рядка
 - Є перевірка помилок введення
 - Використано як `char[]`, так і `std::string`
-

Що можна покращити

1. Перевірка переповнення буфера

У `my_insert()` немає перевірки, чи вистачить місця у `char buffer[1000]`.

Наприклад:

```
if (len + insertLen >= 1000) {
    return;
}
```

2. Використання `size_t`

У функції `my_insert()` краще використовувати `size_t` замість `int` для роботи з довжиною рядків.

3. Обробка некоректного вводу

Якщо користувач введе символ замість числа у меню — програма може працювати неправильно.

1-В 2-В 3-С 4-С 5-С 6-`char[]` — це масив символів із нуль-термінатором `'\0'`, а `std::string` — готовий клас для роботи з рядками. 7- Функція `my_insert()` спочатку визначає довжину основного рядка і рядка для вставки. Потім перевіряє правильність позиції вставки. Далі всі символи основного рядка зсуваються вправо на довжину нового рядка, щоб звільнити місце. Після цього новий рядок копіюється у потрібну позицію. 8-`cin.ignore()` використовується для очищення буфера вводу після `cin >> pos`, щоб символ `Enter` не залишився у потоці вводу і не заважав роботі `getline()`. 9-Можливе переповнення буфера, якщо вставлений рядок буде занадто великий. Також помилки можуть виникнути при неправильному завершенні рядка символом `'\0'` або при виході за межі масиву. 10-Лінійний пошук перевіряє елементи послідовно один за одним і працює на будь-яких масивах. Його складність $O(n)$. Бінарний пошук працює тільки з відсортованими даними, кожного разу ділить масив навпіл і має складність $O(\log n)$. Лінійний пошук доцільний для малих або невідсортованих масивів, а бінарний — для великих відсортованих наборів даних.

Аналіз відповідей та оцінювання

1. Тестові питання

№	Ваша відповідь	Правильність	Аналіз	Оцінка
1	В	✓	Правильно визначено призначення <code>my_insert()</code>	5/5
2	В	✓	Правильно пояснено роль <code>'\0'</code>	5/5
3	С	✓	Вірно вказано <code>rfind()</code>	5/5
4	С	✓	Правильна відповідь про порожній рядок	5/5
5	С	✓	Правильно вказано складність $O(n)$	5/5

Ймовірність використання III:

Низька. Відповіді короткі та природні.

2. Відкриті питання

6. Різниця між `char[]` та `std::string`

Аналіз

Відповідь повна та правильна. Ви пояснили:

- наявність `'\0'`
- автоматичне керування пам'яттю

Що можна було додати

Можна було згадати:

- `char[]` працює на нижчому рівні
- `std::string` є частиною STL

Ймовірність використання III

Середня — відповідь добре структурована та академічна.

Оцінка

5/5 → з урахуванням ймовірності III: 4/5

7. Пояснення роботи `my_insert()`

Аналіз

Дуже хороше покрокове пояснення алгоритму:

- визначення довжин
- перевірка позиції

- зсув символів
- копіювання нового рядка

Правильно пояснено, навіщо потрібен зсув вправо.

Ймовірність використання ШІ

Низька.

Оцінка

5/5 → після корекції: 5/5

8. Для чого `cin.ignore()`

Аналіз

Відповідь повністю правильна. Ви правильно описали:

- очищення буфера
- проблему з Enter
- взаємодію з `getline()`

Ймовірність використання ШІ

Низька.

Оцінка

5/5

9. Можливі помилки у `char[]`

Аналіз

Правильно названі:

- переповнення буфера
- проблема `'\0'`
- вихід за межі масиву

Це ключові ризики роботи з C-рядками.

Ймовірність використання ШІ

Низька-середня.

Оцінка

5/5

10. Лінійний та бінарний пошук

Аналіз

Відповідь дуже хороша:

- описано принцип роботи

- наведено складності
- пояснено сферу застосування

Що можна було додати

Можна було згадати, що бінарний пошук працює швидше саме через поділ масиву навпіл.

Ймовірність використання III

Середня.

Оцінка

5/5 → після корекції: 4/5

Підсумкова таблиця

Питання Бал

1	5
2	5
3	5
4	5
5	5
6	4
7	5
8	5
9	5
10	4

Загальна середня оцінка

Сума балів: **48/50**

Середній бал:

4.8 / 5

Загальний висновок

Рівень підготовки — **добрий**.

Ви:

- розумієте різницю між `char[]` і `std::string`;
- орієнтуєтесь у методах обробки рядків;

- розумієте принцип роботи алгоритмів пошуку;
- можете пояснити власний код.

Найсильніші відповіді:

- `cin.ignore()`
- алгоритм `my_insert()`
- порівняння пошуків

Чому при роботі з C-рядками необхідно контролювати наявність нуль-термінатора і які помилки можуть виникнути при його відсутності?

У C-рядках символ `'\0'` є ознакою завершення рядка. Усі функції роботи з рядками (`strlen`, `strcpy`, `cout` для `char[]` тощо) читають символи доти, поки не зустрінуть нуль-термінатор. Саме тому його наявність обов'язково потрібно контролювати.

Якщо `'\0'` відсутній, програма не розуміє, де закінчується рядок, і починає читати зайву пам'ять за межами масиву. Через це можуть виникати:

- виведення випадкових символів;
- вихід за межі масиву;
- пошкодження даних у пам'яті;
- зависання або аварійне завершення програми;
- помилки безпеки, пов'язані з доступом до чужої області пам'яті.

Наприклад, у моїй роботі після копіювання рядка я вручну додаю:

```
buffer[input.length()] = '\0';
```

Це гарантує правильне завершення C-рядка і коректну роботу функції `my_insert()`.

У лекції розглядаються алгоритми пошуку підрядка КМП, Бойєра-Мура та Рабіна-Карпа. Який із них найбільш ефективний для великих текстів і чому?

Згідно з текстом лекції, найбільш ефективним алгоритмом на практиці є **алгоритм Боєра-Мура (БМ-пошук)**.

Ефективність алгоритму Боєра-Мура:

Умови найкращої роботи: Алгоритм демонструє найвищу ефективність, якщо шуканий зразок (W) довгий, а потужність алфавіту достатньо велика.

Напрямок порівняння: Основна ідея полягає в тому, що порівняння символів відбувається **справа наліво** (починаючи з кінця зразка, а не з його початку).

Механізм зсуву: Після порівняння за допомогою спеціальної евристичної процедури обчислюється величина зсуву вправо. Це дозволяє ігнорувати значні частини тексту.

Кількість порівнянь: У найсприятливіших обставинах кількість порівнянь може становити всього N/M (де N — довжина тексту, а M — довжина зразка).

Порівняння з іншими алгоритмами:

Алгоритм Кнута-Морріса-Пратта (КМП): Хоча він гарантує швидкість порядку $N+M$, на практиці виграш часто незначний, оскільки суттєвий зсув зразка відбувається лише після часткового збігу символів, що в реальних текстах трапляється рідко.

Алгоритм Рабіна-Карпа (РК): Базується на обчисленні хеш-значень (модульної арифметики). Він працює швидко в середньому ($O(N+M)$), але його ефективність залежить від частоти «холостих спрацювань» (колізій), які потребують додаткової посимвольної перевірки.